

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Experiment No.: 5

Student Name: K.Rama

Roll No.: A24126552087

Section: CSM-B

Date: 22-09-2026

1. TITLE

Interactive Product Calculator using HTML, CSS, and JavaScript

2. AIM

To build an interactive product price calculator that computes a total price in real time as the user types, using JavaScript input event listeners, and validates that quantity and price are not negative.

3. OBJECTIVE

The objectives of this experiment are:

- To calculate and update a result live as the user types, using the "input" event instead of a button click.
- To validate numeric input on the client side and display an inline error message without blocking typing.
- To format a numeric result to two decimal places using `toFixed()`.
- To handle empty fields gracefully without showing a false error message.

4. THEORY

Unlike a button-click handler, an "input" event listener fires every time the value of a text or number field changes — on every keystroke, paste, or spin-button click — which is what allows the total price to recalculate live as the user types, without needing a separate "Calculate" button. Converting field values with `Number()` turns the string from the input into a numeric type so arithmetic can be performed; if the field is empty, `Number("")` evaluates to 0 in JavaScript, which is why the code checks `quantity.value === ""` separately from checking for negative values, so an incomplete form is not mistaken for an invalid one. The result is formatted with `toFixed(2)` so that the displayed total always shows exactly two decimal places, matching currency conventions.

5. ALGORITHM / PROCEDURE

1. Create `index.html` with a container holding a heading, labelled inputs for product name, quantity, and price, an empty error paragraph, and a result area showing "Total Price: ₹0.00".
2. Link an external `style.css` file and an external `script.js` file.
3. In `style.css`, style the container as a centred white card, style labels and inputs, and style the result box and the error message (in red).
4. In `script.js`, select the `productName`, `quantity`, `price`, `total`, and `error` elements using `getElementById()`.
5. Write a `calculateTotal()` function that converts `quantity.value` and `price.value` to numbers using `Number()`.
6. Inside `calculateTotal()`, if either numeric value is negative, display an error message, reset the total to "0.00", and return.
7. If either input field is empty, clear any error message, reset the total to "0.00", and return, without treating an empty field as an error.

8. Otherwise, multiply quantity by price, format the result with toFixed(2), display it in the total element, and clear the error message.
9. Attach an "input" event listener to both the quantity and price fields that calls calculateTotal() on every change.
10. Open index.html in a browser and test by typing values into quantity and price, and by entering a negative value.

6. CODE EXPLANATION

index.html defines a card-style container with a heading, three labelled inputs (product name, quantity, price), an empty <p id="error"> for validation messages, and a result block showing the running total inside a . style.css centres the card on the page with Flexbox, gives inputs full-width borders and spacing, styles the result box with a light grey background, and colours the error paragraph red. script.js contains a single calculateTotal() function: it converts the quantity and price fields to numbers, first checking for negative values (showing an error and zeroing the total if either is negative), then checking for empty fields (silently zeroing the total without an error, since an incomplete form isn't itself an error), and finally, if both checks pass, multiplying quantity by price and displaying the result formatted to two decimal places. Because this function is wired to the "input" event on both the quantity and price fields rather than to a button's "click" event, the total recalculates on every keystroke.

7. PROGRAM

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Product Calculator</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <div class="container">
    <h1>Interactive Product Calculator</h1>

    <label for="productName">Product Name</label>
    <input type="text" id="productName" placeholder="Enter product name">

    <label for="quantity">Quantity</label>
    <input type="number" id="quantity" min="0" placeholder="Enter quantity">

    <label for="price">Price per Product</label>
    <input type="number" id="price" min="0" step="0.01" placeholder="Enter price">

    <p id="error"></p>

    <div class="result">
      <h2>Total Price: ₹<span id="total">0.00</span></h2>
    </div>
```

```

    </div>

    <script src="script.js"></script>
</body>
</html>
style.css
body {
    font-family: Arial, sans-serif;
    background-color: #f2f2f2;
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 100vh;
}

.container {
    background-color: white;
    width: 400px;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 0 10px gray;
}

h1 {
    text-align: center;
}

label {
    display: block;
    margin-top: 15px;
    font-weight: bold;
}

input {
    width: 100%;
    padding: 10px;
    margin-top: 5px;
    box-sizing: border-box;
}

.result {
    margin-top: 25px;
    text-align: center;
    background-color: #eeeeee;
    padding: 15px;
}

#error {

```

```

        color: red;
    }
script.js
const productName = document.getElementById("productName");
const quantity = document.getElementById("quantity");
const price = document.getElementById("price");
const total = document.getElementById("total");
const error = document.getElementById("error");

function calculateTotal() {

    const quantityValue = Number(quantity.value);
    const priceValue = Number(price.value);

    if (quantityValue < 0 || priceValue < 0) {
        error.textContent = "Quantity and price cannot be negative.";
        total.textContent = "0.00";
        return;
    }

    if (quantity.value === "" || price.value === "") {
        error.textContent = "";
        total.textContent = "0.00";
        return;
    }

    const totalPrice = quantityValue * priceValue;

    total.textContent = totalPrice.toFixed(2);
    error.textContent = "";
}

quantity.addEventListener("input", calculateTotal);
price.addEventListener("input", calculateTotal);

```

8. TEST CASES AND EXECUTION DATA

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Load page with all fields empty	Total shows "₹0.00" and no error message is displayed	Total and empty error area displayed correctly	Pass
TC-02	Type a quantity and a price	Total price updates live, formatted to 2 decimal places	Total recalculated correctly on every keystroke	Pass
TC-03	Leave quantity or price empty after typing in the other	Total resets to "0.00" and no error message is shown	Total shown as 0.00, error area empty	Pass
TC-04	Enter a negative	Error message	Error message	Pass

	quantity or price	"Quantity and price cannot be negative." is shown and total resets to "0.00"	displayed, total reset to 0.00	
TC-05	Correct a negative value back to a valid positive number	Error message disappears and the total recalculates correctly	Error cleared, correct total displayed	Pass

9. OUTPUT

Initial page load (all fields empty, total ₹0.00):

The screenshot displays the 'Interactive Product Calculator' interface. It features three input fields: 'Product Name' with a placeholder 'Enter product name', 'Quantity' with a placeholder 'Enter quantity' and a spinner control, and 'Price per Product' with a placeholder 'Enter price' and a spinner control. Below these fields, a large grey box displays the 'Total Price: ₹0.00'.

After entering a valid quantity and price:

Interactive Product Calculator

Product Name

Quantity

Price per Product

Total Price: ₹227.50

After entering a negative quantity (validation error shown):

Interactive Product Calculator

Product Name

Quantity

Price per Product

Quantity and price cannot be negative.

Total Price: ₹0.00

10. RESULT

Thus, an interactive product calculator was successfully designed using HTML, CSS, and JavaScript. The total price updates live as the user types, empty fields are handled gracefully, and negative values are correctly validated with an inline error message. The output was verified in a browser.